

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3623 - DATABASE PROGRAMMING 2025/2026 - SEM 1

Project 1 (10%) - Course Registration System (Group 6)

Name	:	MUHAMMAD ADAM BIN RAZALI
Matric Id	:	A23CS0116

Name	:	MUHAMMAD AFIQ DANIAL BIN ROZAIDIE
Matric Id	:	A23CS0117

Name	:	IMAN ABADI BIN MOHD NIZWAN
Matric Id	:	A23CS0084

Name	:	MOHAMED ALIF FATHI BIN ABDUL LATIF
Matric Id	:	A23CS0112



UNIVERSITI TEKNOLOGI MALAYSIA

PRESENTATION SLIDE

PROJECT 1- Course Registration System (Group 6)

Innovating Solutions

www.utm.my

Background and Objective

Background

- This project aims to focus on the **design and the implementation** of a database system for a university Course Registration System. A course registration system is a **process or online platform that allows students to enroll in courses for an academic term**

Objective

- The main objective of this project is to demonstrate how relational databases like MySQL handle relationships between multiple tables by using constraints, indexes and filtering techniques through database management system.

Data Definition Language (DDL)

CREATE DATABASE

```
-- Database creation with the name course_registration  
CREATE DATABASE course_registration;
```

Creates a database named course_registration
using the CREATE DATABASE statement

CREATE TABLE

```
-- Course table creation
CREATE TABLE courses (
  course_id VARCHAR(8) PRIMARY KEY,
  course_code VARCHAR(20) UNIQUE NOT NULL,
  course_title VARCHAR(150) NOT NULL,
  credit_hours INT NOT NULL CHECK(credit_hours > 0),
  capacity INT NOT NULL DEFAULT 40 CHECK(capacity > 0),
  lecturer_id VARCHAR(20),
  prerequisite_id VARCHAR(8) DEFAULT NULL,
  course_desc TEXT,
  FOREIGN KEY (lecturer_id) REFERENCES lecturers(lecturer_id)
    ON UPDATE CASCADE
    ON DELETE SET NULL
);
```

Creates a table named course using the CREATE TABLE statement where the field names are stated as well as the data type of each field with a few constraints.

Constraints Used

```
-- Course table creation
CREATE TABLE courses (
  course_id VARCHAR(8) PRIMARY KEY,
  course_code VARCHAR(20) UNIQUE NOT NULL,
  course_title VARCHAR(150) NOT NULL,
  credit_hours INT NOT NULL CHECK(credit_hours > 0),
  capacity INT NOT NULL DEFAULT 40 CHECK(capacity > 0),
  lecturer_id VARCHAR(20),
  prerequisite_id VARCHAR(8) DEFAULT NULL,
  course_desc TEXT,
  FOREIGN KEY (lecturer_id) REFERENCES lecturers(lecturer_id)
    ON UPDATE CASCADE
    ON DELETE SET NULL
);
```

- PRIMARY KEY
- UNIQUE
- NOT NULL
- CHECK
- DEFAULT
- FOREIGN KEY

ALTER TABLE

```
-- Alter table modifying course_desc column  
ALTER TABLE courses  
MODIFY COLUMN course_desc TEXT NOT NULL;
```

Modifies the course_desc field adding the NOT NULL constraint

DROP TABLE

```
-- Drop prerequisites table  
DROP TABLE IF EXISTS prerequisites;
```

Drops the table named prerequisites in it exists in the database.

Data Manipulation Language (DML)

INSERT STATEMENT

```
INSERT INTO lecturers (lecturer_id, lecturer_fname, lecturer_lname, email, department) VALUES
('L23AC001', 'Farid', 'Hamzah', 'farid.hamzah@university.edu', 'Applied Computing'),
('L23AC002', 'Alicia', 'Tan', 'alicia.tan@university.edu', 'Applied Computing'),
('L23CS001', 'Rahman', 'Yusof', 'rahman.yusof@university.edu', 'Computer Science'),
('L23CS002', 'Nadia', 'Amin', 'nadia.amin@university.edu', 'Computer Science'),
('L23EC001', 'Suhana', 'Ariffin', 'suhana.ariffin@university.edu', 'Emergent Computing'),
('L23SE001', 'Daniel', 'Lim', 'daniel.lim@university.edu', 'Software Engineering'),
('L24AC001', 'Harith', 'Samsul', 'harith.samsul@university.edu', 'Applied Computing'),
('L24CS001', 'Melissa', 'Teh', 'melissa.teh@university.edu', 'Computer Science'),
('L24CS002', 'Arif', 'Kamal', 'arif.kamal@university.edu', 'Computer Science'),
('L24EC001', 'Jasmin', 'Ong', 'jasmin.ong@university.edu', 'Emergent Computing');
```

INSERT STATEMENT

Table Name	No. of Records
students	19 (20 before DELETE statement)
lecturers	10
courses	10
registrations	19

UPDATE WHERE STATEMENT

```
-- Update lecturers table where the lecturer_id is L24CS002  
UPDATE lecturers  
SET department = 'Emergent Computing'  
WHERE lecturer_id = 'L24CS002';
```

DELETE WHERE STATEMENT

```
-- Delete students from students table where student_id is A25CS0020  
DELETE FROM students  
WHERE student_id = 'A25CS0020';
```


Data Query Language (DQL)



1. Filtering

WHERE, AND, OR

```
-- Find students from the 2023 intake who are in Software Engineering
SELECT student_id, student_fname, student_lname, program
FROM students
WHERE intake_year = 2023 AND program = 'Software Engineering';
```

```
-- Students who are in either Data Engineering OR Bioinformatics
SELECT student_id, student_fname, student_lname, program
FROM students
WHERE program = 'Data Engineering' OR program = 'Bioinformatics';
```

Output:

	student_id	student_fname	student_lname	program
▶	A23CS0002	Afiq	Danial	Software Engineering
	A23CS0006	Sarah	Aqilah	Software Engineering

	student_id	student_fname	student_lname	program
▶	A23CS0001	Iman	Nizwan	Data Engineering
	A23CS0005	Hazim	Roslan	Bioinformatics
	A23CS0007	Daniel	Hakim	Data Engineering
	A24CS0009	Mika	Hafiz	Bioinformatics
	A24CS0012	Faris	Azlan	Data Engineering
	A24CS0015	Amani	Najwa	Bioinformatics
	A25CS0017	Sofea	Nurin	Data Engineering

1. Filtering

NOT, BETWEEN

```
-- Courses that are NOT taught by the Applied Computing department
SELECT course_code, course_title, lecturer_id
FROM courses
WHERE lecturer_id NOT IN (
    SELECT lecturer_id
    FROM lecturers
    WHERE department = 'Applied Computing'
);
```

Output:

	course_code	course_title	lecturer_id
▶	CS1013	Introduction to Computer Science	L23CS001
	CS1023	Programming Fundamentals	L23CS002
	CS2013	Object-Oriented Programming	L24CS001
	CS2023	Data Structures and Algorithms	L24CS002
	EC3013	Machine Learning Fundamentals	L23EC001
	SE1013	Introduction to Software Engineering	L23SE001
	CS1033	Database Systems	L24EC001

```
-- Students with intake year between 2023 and 2024
SELECT student_id, student_fname, student_lname, intake_year
FROM students
WHERE intake_year BETWEEN 2023 AND 2024;
```

	student_id	student_fname	student_lname	intake_year
▶	A23CS0001	Iman	Nizwan	2023
	A23CS0002	Afiq	Danial	2023
	A23CS0003	Alif	Fathi	2023
	A23CS0004	Irfan	Shaqir	2023
	A23CS0005	Hazim	Roslan	2023
	A23CS0006	Sarah	Aqilah	2023
	A23CS0007	Daniel	Hakim	2023
	A23CS0008	Alya	Suhana	2023

1. Filtering

LIKE, NULL

```
-- Students whose first name starts with 'A'
SELECT student_id, student_fname, student_lname
FROM students
WHERE student_fname LIKE 'A%';
```

```
-- Courses that do NOT have prerequisites
SELECT course_id, course_code, course_title
FROM courses
WHERE prerequisite_id IS NULL;
```

Output:

	student_id	student_fname	student_lname
▶	A23CS0002	Afiq	Danial
	A23CS0003	Alif	Fathi
	A23CS0008	Alya	Suhana
	A24CS0015	Amani	Najwa

	course_id	course_code	course_title
▶	C0000001	CS1013	Introduction to Computer Science
	C0000005	AC1013	Discrete Mathematics
	C0000008	SE1013	Introduction to Software Engineering

2. Sorting

ORDER BY, LIMIT

-- Most recent registrations first

```
SELECT reg_id, student_id, course_id, reg_date
FROM registrations
ORDER BY reg_date DESC;
```

Output:

	reg_id	student_id	course_id	reg_date
▶	18	A25CS0016	C0000001	2025-09-10
	19	A25CS0019	C0000005	2025-09-10
	17	A24CS0010	C0000009	2025-07-16
	4	A23CS0001	C0000003	2025-02-17
	15	A24CS0009	C0000002	2025-01-15
	16	A24CS0010	C0000008	2025-01-10
	6	A23CS0002	C0000008	2024-08-16
	9	A23CS0003	C0000004	2024-07-17

-- Top 5 newest students

```
SELECT student_id, student_fname, student_lname, intake_year
FROM students
ORDER BY intake_year DESC
LIMIT 5;
```

	student_id	student_fname	student_lname	intake_year
▶	A25CS0017	Sofea	Nurin	2025
	A25CS0016	Rayan	Hakimi	2025
	A25CS0019	Maisarah	Ismail	2025
	A25CS0018	Izz	Haikal	2025
	A24CS0012	Faris	Azlan	2024

3. Aggregation

COUNT, SUM, AVG

```
-- Total number of students in each program
```

```
SELECT program, COUNT(*) AS total_students
```

```
FROM students
```

```
GROUP BY program;
```

```
-- Total credit hours a student registered in a semester
```

```
SELECT student_id, semester, SUM(credit_hours) AS total_credits
```

```
FROM registrations r
```

```
JOIN courses c ON r.course_id = c.course_id
```

```
GROUP BY student_id, semester;
```

```
-- Average course capacity across all courses
```

```
SELECT AVG(capacity) AS avg_capacity
```

```
FROM courses;
```

Output:

	program	total_students
▶	Data Engineering	4
	Software Engineering	4
	Networks and Security	4
	Graphics and Multimedia	4
	Bioinformatics	3

	student_id	semester	total_credits
▶	A23CS0001	23/24-1	6
	A23CS0002	23/24-1	3
	A23CS0003	23/24-1	3
	A23CS0004	23/24-1	3
	A24CS0009	24/25-1	3
	A25CS0016	25/26-1	3
	A23CS0001	23/24-2	3
	A23CS0003	23/24-2	3

	avg_capacity
▶	36.0000

3. Aggregation

MAX, MIN

```
-- Course with the highest capacity
SELECT course_code, course_title, capacity
FROM courses
WHERE capacity = (SELECT MAX(capacity) FROM courses);
```

```
-- Course with the smallest number of credit hours
SELECT course_code, course_title, credit_hours
FROM courses
WHERE credit_hours = (SELECT MIN(credit_hours) FROM courses);
```

Output:

	course_code	course_title	capacity
▶	SE2023	Software Design and Architecture	50

	course_code	course_title	credit_hours
▶	CS1013	Introduction to Computer Science	3
	CS1023	Programming Fundamentals	3
	CS2013	Object-Oriented Programming	3
	CS2023	Data Structures and Algorithms	3
	AC1013	Discrete Mathematics	3
	AC2023	Computational Mathematics	3
	EC3013	Machine Learning Fundamentals	3
	SE1013	Introduction to Software Engineering	3

4. Grouping and filtering groups

```
-- Count of lecturers for each departments which exceeds 2
SELECT department, COUNT(lecturer_id) AS total_lecturer
FROM lecturers
GROUP BY department
HAVING total_lecturer > 2;
```

Output:

department	total_lecturer
Applied Computing	3
Computer Science	3
Emergent Computing	3

5. Numeric and string functions

```
-- List students with only short name
SELECT
  student_id,
  UPPER(CONCAT(student_fname, ' ', student_lname)) AS full_name,
  SUBSTR(student_id, 2, 2) AS year_intake,
  LENGTH(CONCAT(student_fname, ' ', student_lname)) AS name_count
FROM students
WHERE LENGTH(CONCAT(student_fname, ' ', student_lname)) > 12;
```

Output:

student_id	full_name	year_intake	name_count
A23CS0001	IMAN NIZWAN	23	11
A23CS0002	AFIQ DANIAL	23	11
A23CS0003	ALIF FATHI	23	10
A23CS0008	ALYA SUHANA	23	11
A24CS0009	MIKA HAFIZ	24	10
A24CS0010	SYAFIQ AMIR	24	11
A24CS0012	FARIS AZLAN	24	11
A24CS0013	NADIA SOFEA	24	11

5. Numeric and string functions

```
-- Monitor capacity of the class for each course
SELECT course_code, capacity,
ROUND(capacity * 0.85, 2) AS warning_level_rounded,
TRUNCATE(capacity * 0.85, 2) AS warning_level_truncated
FROM courses;
```

Output:

course_code	capacity	warning_level_rounded	warning_level_truncated
CS1013	40	34.00	34.00
CS1023	30	25.50	25.50
CS2013	40	34.00	34.00

6. Conditional logic

```
-- Classify students by seniority based on intake year
```

```
SELECT
```

```
  student_id,  
  intake_year,
```

```
  CASE
```

```
    WHEN intake_year = 2023 THEN 'Senior'
```

```
    WHEN intake_year = 2024 THEN 'Junior'
```

```
    WHEN intake_year = 2025 THEN 'Freshman'
```

```
    ELSE 'Unknown'
```

```
  END AS student_year_group
```

```
FROM students;
```

Output:

A23CS0007	2023	Senior
A23CS0008	2023	Senior
A24CS0009	2024	Junior
A24CS0010	2024	Junior
A24CS0011	2024	Junior
A24CS0012	2024	Junior
A24CS0013	2024	Junior
A24CS0014	2024	Junior
A24CS0015	2024	Junior
A25CS0016	2025	Freshman

7.1. Subqueries - Single Row

```
-- Single Row : Find students who intake in very recent year
SELECT student_id, student_fname, student_lname, email, program, intake_year
FROM students
WHERE intake_year = (
    SELECT MAX(intake_year)
    FROM students
);
```

Output:

	student_id	student_fname	student_lname	email	program	intake_year
▶	A25CS0016	Rayan	Hakimi	rayan.hakimi@cs.edu	Software Engineering	2025
	A25CS0017	Sofea	Nurin	sofea.nurin@cs.edu	Data Engineering	2025
	A25CS0018	Izz	Haikal	izz.haikal@cs.edu	Networks and Security	2025
	A25CS0019	Maisarah	Ismail	maisarah.ismail@cs.edu	Graphics and Multimedia	2025
●	NULL	NULL	NULL	NULL	NULL	NULL

7.2. Subqueries - Multiple Row

```
-- Multiple Row : Find course who teach by a lecturer from Applied Computing department
SELECT lecturer_id, course_id, course_code, course_title
FROM courses
WHERE lecturer_id IN (
  SELECT lecturer_id
  FROM lecturers
  WHERE department = 'Applied Computing'
);
```

Output:

	lecturer_id	course_id	course_code	course_title
▶	L23AC001	C0000005	AC1013	Discrete Mathematics
	L23AC002	C0000006	AC2023	Computational Mathematics
	L24AC001	C0000009	SE2023	Software Design and Architecture
✱	NULL	NULL	NULL	NULL

7.3. Subqueries - Correlated

-- Correlated : Find students who register more than 2 courses

```
SELECT r1.student_id, r1.course_id, r1.semester
FROM registrations r1
WHERE (
  SELECT COUNT(*)
  FROM registrations
  WHERE r1.student_id = student_id
) > 2;
```

Output:

	student_id	course_id	semester
▶	A23CS0001	C0000001	23/24-1
	A23CS0001	C0000002	23/24-2
	A23CS0001	C0000003	24/25-1
	A23CS0001	C0000005	23/24-1
	A23CS0003	C0000001	23/24-1
	A23CS0003	C0000002	23/24-2
	A23CS0003	C0000004	24/25-2

8.1. Set Operations - UNION

```
-- UNION : List all courses that are 3 credit hours or have a capacity more than 40
SELECT course_id, course_title, credit_hours
FROM courses
WHERE credit_hours = 3
UNION
SELECT course_id, course_title, credit_hours
FROM courses
WHERE capacity > 40;
```

Output:

	course_id	course_title	credit_hours
▶	C0000001	Introduction to Computer Science	3
	C0000002	Programming Fundamentals	3
	C0000003	Object-Oriented Programming	3
	C0000004	Data Structures and Algorithms	3
	C0000005	Discrete Mathematics	3
	C0000006	Computational Mathematics	3
	C0000007	Machine Learning Fundamentals	3
	C0000008	Introduction to Software Engineering	3
	C0000009	Software Design and Architecture	3
	C0000010	Database Systems	3

8.2. Set Operations - NOT EXIST

```
-- NOT EXIST : Find student who doesnt enroll in course of Introduction to Computer Science - C0000001
SELECT s.student_id, CONCAT(s.student_fname, ' ', s.student_lname) AS student_fullname
FROM students s
WHERE NOT EXISTS (
  SELECT 1
  FROM registrations r
  WHERE r.student_id = s.student_id
    AND r.course_id = 'C0000001'
);
```

Output:

	student_id	student_fullname
▶	A23CS0005	Hazim Roslan
	A23CS0006	Sarah Aqilah
	A23CS0007	Daniel Hakim
	A23CS0008	Alya Suhana
	A24CS0010	Syafiq Amir
	A24CS0011	Hannah Zulaikha
	A24CS0012	Faris Azlan
	A24CS0013	Nadia Sofea
	A24CS0014	Zharif Imran
	A24CS0015	Amani Najwa
	A25CS0017	Sofea Nurin
	A25CS0018	Izz Haikal
	A25CS0019	Maisarah Ismail

9.1. Joins - NATURAL

```
-- NATURAL : All list registration with students name
SELECT student_id, CONCAT(student_fname, ' ', student_lname) AS fullname,
       course_id, semester
FROM registrations
NATURAL JOIN students;
```

Output:

	student_id	fullname	course_id	semester
▶	A23CS0001	Iman Nizwan	C0000001	23/24-1
	A23CS0001	Iman Nizwan	C0000002	23/24-2
	A23CS0001	Iman Nizwan	C0000003	24/25-1
	A23CS0001	Iman Nizwan	C0000005	23/24-1
	A23CS0002	Afiq Danial	C0000001	23/24-1
	A23CS0002	Afiq Danial	C0000008	23/24-2
	A23CS0003	Alif Fathi	C0000001	23/24-1
	A23CS0003	Alif Fathi	C0000002	23/24-2
	A23CS0003	Alif Fathi	C0000004	24/25-2
	A23CS0004	Irfan Shaqir	C0000001	23/24-1
	A23CS0004	Irfan Shaqir	C0000010	23/24-2
	A23CS0005	Hazim Roslan	C0000005	23/24-1

9.2. Joins - INNER

```
-- INNER : List all courses with assigned lecturer
SELECT  c.course_code, c.course_title,
        UPPER(CONCAT(l.lecturer_fname, ' ', l.lecturer_lname)) AS lecturer_fullname,
        l.lecturer_id
FROM    courses c
INNER JOIN lecturers l
ON      c.lecturer_id = l.lecturer_id;
```

Output:

	course_code	course_title	lecturer_fullname	lecturer_id
►	CS1013	Introduction to Computer Science	RAHMAN YUSOF	L23CS001
	CS1023	Programming Fundamentals	NADIA AMIN	L23CS002
	CS2013	Object-Oriented Programming	MELISSA TEH	L24CS001
	CS2023	Data Structures and Algorithms	ARIF KAMAL	L24CS002
	AC1013	Discrete Mathematics	FARID HAMZAH	L23AC001
	AC2023	Computational Mathematics	ALICIA TAN	L23AC002
	EC3013	Machine Learning Fundamentals	SUHANA ARIFFIN	L23EC001
	SE1013	Introduction to Software Engineering	DANIEL LIM	L23SE001
	SE2023	Software Design and Architecture	HARITH SAMSUL	L24AC001
	CS1033	Database Systems	JASMIN ONG	L24EC001

9.3. Joins - LEFT OUTER

```
-- LEFT OUTER : List all courses and the students registered (if any)
SELECT c.course_id, c.course_title, s.student_id,
       LOWER(CONCAT(s.student_fname, ' ', s.student_lname)) AS fullname_student
FROM courses c
LEFT OUTER JOIN registrations r
ON c.course_id = r.course_id
LEFT OUTER JOIN students s
ON r.student_id = s.student_id;
```

Output:

	course_id	course_title	student_id	fullname_student
►	C0000001	Introduction to Computer Science	A23CS0001	iman nizwan
	C0000001	Introduction to Computer Science	A23CS0002	afiq danial
	C0000001	Introduction to Computer Science	A23CS0003	alif fathi
	C0000001	Introduction to Computer Science	A23CS0004	irfan shaqir
	C0000001	Introduction to Computer Science	A24CS0009	mika hafiz
	C0000001	Introduction to Computer Science	A25CS0016	rayan hakimi
	C0000002	Programming Fundamentals	A23CS0001	iman nizwan
	C0000002	Programming Fundamentals	A23CS0003	alif fathi
	C0000002	Programming Fundamentals	A24CS0009	mika hafiz
	C0000003	Object-Oriented Programming	A23CS0001	iman nizwan
	C0000004	Data Structures and Algorithms	A23CS0003	alif fathi

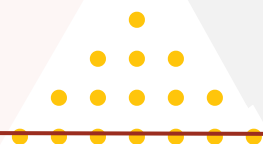
9.4. Joins - SELF JOIN

```
-- SELF : List of courses with its prerequisite courses
SELECT c.course_id, c.course_title, p.course_id, p.course_title
FROM courses c
LEFT JOIN courses p
  ON c.prerequisite_id = p.course_id;
```

Output:

	course_id	course_title	course_id	course_title
▶	C0000001	Introduction to Computer Science	NULL	NULL
	C0000002	Programming Fundamentals	C0000001	Introduction to Computer Science
	C0000003	Object-Oriented Programming	C0000002	Programming Fundamentals
	C0000004	Data Structures and Algorithms	C0000003	Object-Oriented Programming
	C0000005	Discrete Mathematics	NULL	NULL
	C0000006	Computational Mathematics	C0000005	Discrete Mathematics
	C0000007	Machine Learning Fundamentals	C0000004	Data Structures and Algorithms
	C0000008	Introduction to Software Engineering	NULL	NULL
	C0000009	Software Design and Architecture	C0000008	Introduction to Software Engineering
	C0000010	Database Systems	C0000001	Introduction to Computer Science

Indexing (B-tree and TEXT)



Before B-Tree Indexing

Query:

Execution Time:

```
SELECT *
FROM registrations
WHERE reg_date BETWEEN '2023-01-01' AND '2023-12-31';

SET profiling = 1;

SHOW PROFILES;
```

0.00182650

SELECT * FROM registrations WHER...

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	registrations	NULL	ALL	NULL	NULL	NULL	NULL	19	11.11	Using where

After B-Tree Indexing

Create B-Tree index

```
CREATE INDEX idx_reg_date
ON registrations (reg_date);
```

Execution Time:

0.00040350 SELECT * FROM registrations WHER...

approximately 4.5 times faster

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	registrations	NULL	range	idx_reg_date	idx_reg_date	4	NULL	6	100.00	Using index condition

TEXT Indexing

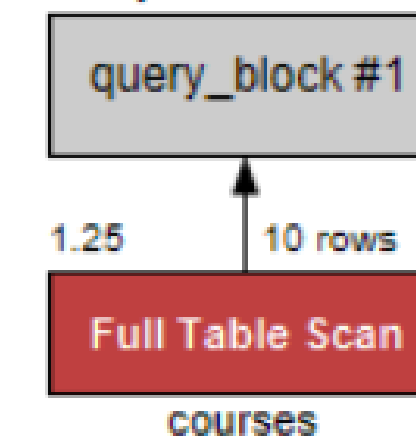
Before Text Indexing

Execution Time:

```
EXPLAIN SELECT *
FROM courses
WHERE course_desc LIKE '%classification%';
```

```
SELECT *
FROM courses
WHERE course_desc LIKE '%classification%';
```

Query cost: 1.25



	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	courses	NULL	ALL	NULL	NULL	NULL	NULL	10	11.11	Using where

	course_id	course_code	course_title	credit_hours	capacity	lecturer_id	prerequisite_id	course_desc
►	C0000007	EC3013	Machine Learning Fundamentals	3	40	L23EC001	C0000004	Core ML algorithms including classification, regr...
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

61	0.03502325	EXPLAIN SELECT * FROM courses WHERE cour...
62	0.01006075	SELECT * FROM courses WHERE course_desc L...

After Text Indexing

Create TEXT index

```
CREATE FULLTEXT INDEX idx_course_desc ON courses(course_desc);
```

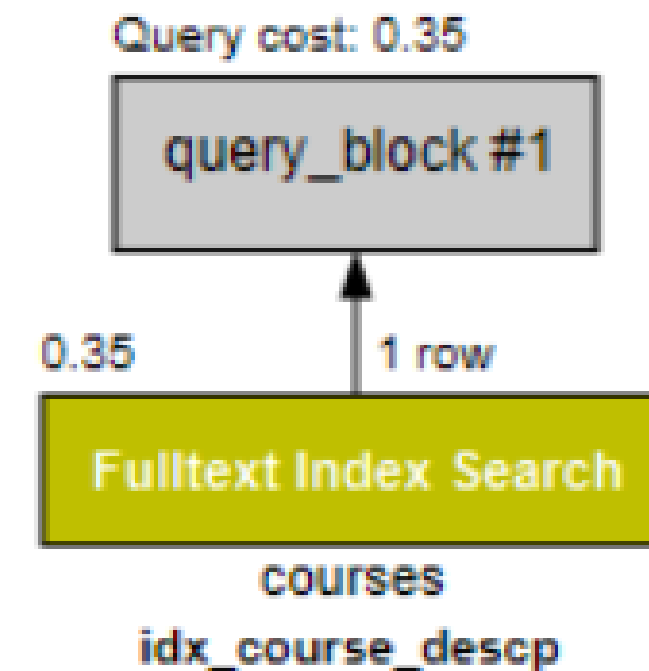
```
EXPLAIN SELECT *
FROM courses
WHERE MATCH(course_desc) AGAINST('classification' IN NATURAL LANGUAGE MODE);
```

```
SELECT *
FROM courses
WHERE MATCH(course_desc) AGAINST('classification' IN NATURAL LANGUAGE MODE);
```

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	courses	NULL	fulltext	idx_course_desc	idx_course_desc	0	const	1	100.00	Using where; Ft_hints: sorted

	course_id	course_code	course_title	credit_hours	capacity	lecturer_id	prerequisite_id	course_desc
►	C0000007	EC3013	Machine Learning Fundamentals	3	40	L23EC001	C0000004	Core ML algorithms including classification, regr...
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Execution Time:



84	0.00241650	EXPLAIN EXPLAIN SELECT * FROM courses WH...
85	0.00133325	SELECT * FROM courses WHERE MATCH(course...

Team Role - Adam

- **Advanced SQL Mastery:** Learned complex data retrieval techniques, including aggregation, grouping, string/numeric functions, and conditional logic using CASE.
- **Database Optimization:** Gained a deep understanding of performance optimization by implementing B-tree indexing and observing how it reduces query execution time.
- **Professional Development:** Enhanced overall technical skills in database management systems while improving team collaboration and organizational efficiency.

Team Role - Afiq Danial

I was responsible for assigning roles to each team member and compiling the conclusion section of the project report. Additionally, I developed data retrieval SQL queries for Part C, covering subqueries, set operations, and various types of joins. I also explained how the use of a TEXT index changes key usage and impacts query performance.

Team Role - Iman Abadi

I was responsible for executing the DML and DDL questions and designing the database that is logical. I learned how different SQL constraints, such as primary keys, foreign keys, unique, check, not null, and default, help maintain data accuracy and consistency in a database. I was also responsible for handling the TEXT indexing and gained experience applying indexing and understanding query performance differences between LIKE searches and FULLTEXT indexing.

Team Role - Alif Fathi

- Handled Part C Question 1, 2, and 3, which involved writing SQL queries using filtering, sorting, grouping, and aggregation.
- Created a B-Tree index and learned how indexing improves performance by speeding up searches and reducing the amount of data scanned.

THANK YOU



univteknologimalaysia



utm.my

utmofficial